

Config-Driven Multi-Robot ROS 2 / Nav2

Bu düzeltilmiş sürüm, sağlanan PDF ve ilk düzeltme notunu ve ikinci turdaki yedi maddeyi temel alır. Mimari sözleşmesini tanımlar; uygulama kodunun değiştirilmiş veya çalıştırılarak doğrulanmış olduğunu ifade etmez. Dört belge aynı config, zaman ve API kararlarını kullanır.

1. Korunan mimari

Robot kimlikleri konfigürasyondan gelir; sabit robot sayısı yoktur. `robot_count = len(config.robots)`. Her robot için model spawn, `robot_state_publisher`, lokalizasyon/Nav2 ve `battery_simulator` başlatılır. Ortak `map_server`, `/map` ve Fleet Manager kullanılır.

Robot türü model adaptöründe, sensör eşlemelerinde ve Nav2 profilinde çözülür. Fleet Manager kimlik, pose, SoC ve operasyon durumuyla çalışır. Yeni robot tipi model varlıkları ve adaptör gerektirebilir; desteklenen tipler backend kataloğundan seçilir.

2. Tek kalıcı konfigürasyon

```
workspace/  
  config/fleet.yaml    # tek düzenlenebilir kaynak  
  fleet_sim/           # world, URDF, launch, adaptörler  
  fleet_manager/       # domain, application, API, repository  
  battery_simulator/  
  fleet_interfaces/  
  frontend/
```

`fleet.yaml` üst bölümleri: `fleet`, `battery`, `assignment`, `tasks`, `navigation`, `simulation`, `robots`. `nav2_profile` adları `navigation.profiles`, `battery_model` adları `battery.models` içinde çözülür. `battery.yaml`, `assignment.yaml` ve `robots/*.yaml` şeklinde bağımsız düzenlenebilir kaynaklar kullanılmaz.

Fleet Manager ve `fleet_sim` aynı mutlak config yolunu okur. Nav2/ROS için gereken parametre dosyaları bu dosyadan türetilen çalışma zamanı çıktılarıdır; elle düzenlenmez. URDF, harita ve dünya dosyaları varlık olarak ayrı kalabilir. SQLite görev, olay, deney ve audit verisini tutar; ikinci config otoritesi değildir.

Şema ve doğrulama

3. Örnek fleet.yaml özet

```

fleet: {name: demo}
battery:
models:
  standard: {idle_drain_per_sec: 0.00001,
    moving_drain_per_sec: 0.00010, charge_per_sec: 0.00050,
    consumption_per_meter: 0.0}
policy: {emergency_level: 0.08, low_level: 0.25,
  idle_charge_level: 0.50, charge_release_level: 0.80, safety_reserve: 0.10}
assignment:
  strategy: CostBased
  weights: {wd: 0.35, we: 0.30, wt: 0.20, wb: 0.15, wl: 0.0}
tasks: {loading_duration: 5.0, unloading_duration: 5.0,
  max_navigation_retries: 2}
navigation:
  navigation_timeout: 120.0
profiles:
  default: {goal_tolerance: {xy: 0.25, yaw: 0.25}}
simulation: {use_sim_time: true}
robots:
  - id: picker_left
    type: turtlebot3_burger
    spawn: {x: 2.5, y: 4.0, yaw: 1.57}
    nav2_profile: default
    battery_model: standard
    initial_percentage: 0.80

```

Bu özet şema örneğidir; Nav2 adaptörü profile alanlarını seçilen dağıtımın goal checker parametrelerine dönüştürür. x/y metre, yaw radyandır. Harita, dünya, istasyonlar ve adlandırılmış lokasyonlar fleet/simulation bölümlerinde tanımlanır.

Doğrulama	Kural
Kimlik ve referans	ROS namespace uyumlu benzersiz id; desteklenen type; çözülen profil. Robot id değişmez; yeniden adlandırma sil+ekle işlemi.
Spawn	Sonlu koordinatlar; dünya sınırı, footprint ve güvenlik açıklığıyla engel/robot çakışma kontrolü.
Politika ve strateji	$0 \leq \text{emergency} < \text{low} < \text{idle_charge_level} \leq \text{release} \leq 1$; $\text{emergency} \leq \text{reserve} \leq 1$; ağırlıklar ≥ 0 ve toplam 1 (sayısal tolerans $1e-6$).
Süre ve model	Süreler sonlu ve ≥ 0 ; timeout > 0 ; retry tam sayı ≥ 0 ; model katsayıları ≥ 0 .

Namespace, TF ve başlatma

4. Robot izolasyonu

```

/<id>/scan          /<id>/odom
/<id>/cmd_vel        /<id>/battery_state
/<id>/pose           /<id>/dock_contact
/<id>/navigate_to_pose /<id>/lifecycle_manager
/<id>/planner_server  /<id>/controller_server
/<id>/behavior_server /<id>/bt_navigator
Ortak: /map, /map_server, /fleet_manager

```

```

map -> picker_left/odom -> picker_left/base_link
map -> delivery_a/odom -> delivery_a/base_link

```

Namespace, TF frame_id alanını otomatik değiştirmez. URDF, odometri, sensörler, lokalizasyon ve Nav2 aynı benzersiz frame sözleşmesini kullanır. Ortak map frame korunur; gerekli model topic eşlemeleri adaptörde yapılır.

5. Dinamik launch sırası

```

cfg = validate(load(shared_fleet_yaml))
for robot in cfg.robots:
    spawn_model(robot.type, robot.spawn)
    start_robot_state_publisher(robot.id)
    start_localization_and_nav2(robot.id, resolved_nav2_profile)
    start_battery_simulator(robot.id, resolved_battery_model)
    wait_for_tf_topics_actions_and_lifecycle()
    register_robot_as_unavailable_until_reconciled(robot.id)

```

Örnek kavramsaldır. Hazırlık kontrolleri sonlu timeout ile çalışır. Pose/SoC taze, TF geçerli ve Nav2 action hazır olmadan robot göreve uygun sayılmaz. Deney reseti için bu hazırlık bariyeri de geçilir.

MVP ve kapsam dışı

Webden ekle/sil/düzenle desteklenir; robot seti değişikliği kontrollü Restart/Apply ile uygulanır. Hot-add/hot-remove, lifecycle, TF, entity ve görev temizliğiyle birlikte ayrı geliştirmedir. Trafik rezervasyonu ve ortak çarpışmasız rota planlama bu atama mimarisinin sağladığı bir garanti değildir.

Taslak API ve sürüm sözleşmesi

6. Taslak düzenleme

Uç	Sonuç
GET /api/config	Taslak içerik, ETag ve üç revision; alanların Live/Restart sınıfı.
PATCH /api/config	Skaler/nesne ayarlarını taslakta günceller; 200 ve yeni ETag. robots alanı reddedilir (422).
GET /api/config/robots	Taslak robot listesi; 200.
POST /api/config/robots	Taslağa robot ekle; 201.
PUT /api/config/robots/{id}	Tam robot tanımını güncelle; 200.
DELETE /api/config/robots/{id}	Taslak robot kaydını kaldır; 204. Geçmiş istatistikler kalır.
POST /api/config/apply	Taslağı doğrula ve uygula; 202 + operation_id.

Bütün mutasyonlar ve Apply, If-Match: "<pending_revision>" gerektirir. Eksik başlık 428; eski revision 412; kimlik/durum çatışması 409; bulunamayan kayıt 404; geçersiz alan 422. PATCH için merge-patch kullanılır; robots dizisinin tümünden değiştirilmesi desteklenmez. Başarılı her taslak değişimi yeni opak revision üretir.

7. Üç revision

Alan	Anlam
pending_revision	Sunucu belleğindeki düzenleme taslağı; restartta kaydedilmemiş taslak kaybolur.
active_revision	Atomik kaydedilmiş fleet.yaml sürümü; kabul edilmiş hedef yapılandırma.
running_revision	Çalışan bileşenlerin başarıyla etkinleştirdiği yapılandırma.

Changes pending = pending != active OR active != running. Applied ancak üçü eşit ve işlem başarılıysa gösterilir. Configured robots taslak uzunluğudur; Running robots runtime registry sayısıdır. Aynı robot sayısı alan değişimi de pending üretir. Başarısız Apply ayrıca Failed gösterir; running_revision eski kalabilir veya runtime yoksa null olabilir.

Atomik kayıt ve Apply

8. Tek yazıcı ve güvenli uygulama

ConfigurationService aynı state sahibi üzerinden seri çalışır. CRUD yalnız taslağı değiştirir. Apply, Live değişiklikleri de kapsayan tek etkinleştirme kapısıdır: yalnız Live alanlar değişmişse yeniden başlatma gerekmez; en az bir Restart alanı varsa tam kontrollü restart yapılır.

1. If-Match ve çalışma modu kontrol edilir. Taslak bütünü doğrulanır; apply kilidi alınır ve config_apply_started yayınlanır. İşlem boyunca yeni taslak değişimi ve komutlar 409 applying alır; servis kullanılamıyorsa 503 applying döner.

2. Yalnız Restart sınıfı değişiklik içeren Apply, aktif teslimat veya ONBOARD/UNKNOWN yük varken 409 döner. Yalnız Live değişiklik içeren Apply aktif teslimat sırasında kabul edilir; ayarlar tanımlı karar sınırında etkinleşir. MVP zorla durdurup yük kaydını silmez; görev bitişi veya operatör çözümü beklenir. Şarj/navigasyon kontrollü sonlandırılır; duruş doğrulanmadan restart ilerlemez.

3. Hedef aynı dizinde geçici dosyaya yazılır, flush/fsync yapılır ve os.replace ile fleet.yaml yerine geçirilir; gerektiğinde izin fsync uygulanır. Tek dosya değişimi atomiktir; dosya + SQLite + ROS restart birlikte atomik değildir. İşlem kaydı ve aşamalarla recovery yapılır.

4. active_revision hedef olur. Gerekirse simülasyon/bileşenler kapatılır; aynı dosya okunarak N robot, N Nav2, N simülatör ve registry kurulur. Restart-Apply QUEUED görevleri SQLite içinde korunur; registry kurulmadan önce manager_epoch yenilenir ve eski sonuçlar elenir. Korunan görevlerin lokasyon/frame referansları yeni config ile doğrulanır; uyumsuz değişiklik Apply öncesi 422 ile reddedilir. Live Apply yalnız yeni karar sınırında parametreleri devreye alır. config_apply_progress aşama bilgisini taşır.

5. Hazırlık kontrolünden sonra running_revision güncellenir, config_applied yayınlanır. Robot setini etkileyen restart sonunda bağlantılara resync_required gönderilir; panel yeni snapshot alır. Config_apply_failed halinde hata ve aşama kaydedilir, Applied gösterilmez.

Hata ve yeniden başlatma

Son başarılı config içeriği kontrollü kurtarma kopyası olarak saklanır; bağımsız düzenlenebilir kaynak değildir. Otomatik sessiz rollback yapılmaz. Açılıştaki yarım işlem fark edilirse atama kilitli başlar, kalıcı config ve runtime uzlaştırılır. Yeniden deneme veya doğrulanmış geri dönüş yeni operation olarak kayıtlı yürütülür.

Live sınırları ve kabul ölçütleri

Live: Apply sonrası yeni sınırdaki	Restart/Apply
Strategy, wd/we/wt/wb/wl; navigation_timeout; görev süreleri ve retry limiti.	Robot ekle/sil, type, spawn, initial_percentage, nav2_profile.
emergency_level, low_level, idle_charge_level, charge_release_level, safety_reserve.	battery_model ve model katsayıları; simülasyon/world ayarları.
Yeni atama/goal/aşama/şarj oturumu yeni sürümü yakalar. Aktif timer, retry dizisi ve şarj oturumu değişmez.	navigation.profiles.*.goal_tolerance: Nav2 goal checker ayarıdır; Fleet Manager Live ayarı değildir.

Aktif şarj oturumu kendi policy_revision değerini taşır. Oturum dışındaki yeni politika kararları yeni sürümü kullanır; çalışan timeout ve retry bütçesi geriye dönük değiştirilmez. UI bu gecikmeli etkinleşme sınırını gösterir.

9. Kabul kontrolleri

2, 4 ve 7 robot tanımı kod değişmeden aynı sayıda instance üretmeli; özel id ve model profilleri çözülmelidir. Topic/action/TF çakışması bulunmamalıdır. Geçersiz spawn ve profil backend tarafından reddedilmelidir.

İki istemcinin aynı ETag ile düzenlemesinde yalnız ilki başarılı olmalı. Robot dizisi PATCH ile değiştirilememeli. Kaydetme sırasında kesinti eski veya yeni tam dosya bırakmalı; yarım dosya kalmamalıdır.

Restart gerektiren alanla Live alan birlikteyse tek Restart/Apply akışı çalışmalı. Apply hatası active/running farkını görünür tutmalı. Silinen robotun tarihsel görevleri ve istatistikleri korunmalıdır. Deney sırasında config değişiklikleri kilitli olmalıdır.

Kaynak: config_driven_multi_robot_ros2_nav2_architecture_tr_v1_1.pdf (6 sayfa). Bu v1.3 FIX, tek dosya, revision, Apply ve alan sınıflamasını netleştirir. Atomik değiştirme referansı: <https://docs.python.org/3/library/os.html#os.replace>

Eksiksiz parametre şeması / 1

Aşağıdaki alanlar önceki özet fleet.yaml şemasının devamıdır; ikinci config dosyası oluşturmaz. Yinelenen üst anahtarlar tek ağaçta birleştirilir. Değerler simülasyon başlangıç önerileridir.

```
assignment:
  uncertainty: 0.15
  references: {D_ref: 20.0, S_ref: 0.20, T_ref: 120.0, L_ref: 600.0}
  workload_window: 3600.0
tasks:
  retry_backoff: 2.0
  max_requeues: 2
  requeue_cooldown: 10.0
navigation:
  recovery_cooldown: 10.0
battery:
  policy:
    idle_charge_level: 0.50
    dock_timeout: 30.0
    charge_timeout: 3600.0
docking:
  stationary_linear_threshold: 0.02
  stationary_angular_threshold: 0.05
  stable_duration: 1.0
fleet:
  health:
    robot_heartbeat_interval: 1.0
    robot_heartbeat_timeout: 5.0
    pose_stale_after: 2.0
    battery_stale_after: 2.0
    dock_contact_stale_after: 2.0
  web:
    heartbeat_interval: 5.0
    heartbeat_timeout: 15.0
    telemetry_publish_hz: 5.0
    replay_capacity: 1000
    send_queue_capacity: 256
```

D_ref metre; S_ref SoC kesri; T_ref/L_ref ve workload_window simülasyon saniyesidir.

Görev/backoff/cooldown/dock/charge süreleri /clock ile çalışır. Health ve Web heartbeat/bayatlık süreleri monotonic duvar saatidir. Simülasyon pause halinde kontrol deadline ilerlemez; UI bayatlık ve pause durumunu birlikte gösterir.

Robot heartbeat üreticisi simulation_adapter içindeki liveness timerıdır: /<id>/heartbeat, std_msgs/msg/Empty; robot bileşenlerinin canlılığını izler. Bu bir güvenli atama kanıtı değildir: pose/SoC/Nav2 hazır kontrolü ayrıca gerekir. Timer simülasyon pause sırasında duvar saatiyle sürer.

Eksiksiz parametre şeması / 2

fleet:

```
map: {frame_id: map, image: maps/warehouse.png,
      resolution: 0.05, origin: {x: 0.0, y: 0.0, yaw: 0.0}}
```

stations:

```
- id: charger_a
  capacity: 1
  dock_pose: {x: 1.0, y: 1.0, yaw: 0.0, frame_id: map}
  zone: {shape: circle, radius: 0.30}
  contact_radius: 0.10
  contact_yaw_tolerance: 0.15
```

locations:

```
- id: pickup_a
  name: Alım A
  kind: pickup
  pose: {x: 2.0, y: 2.0, yaw: 0.0, frame_id: map}
- id: dropoff_a
  name: Teslim A
  kind: dropoff
  pose: {x: 4.0, y: 2.0, yaw: 0.0, frame_id: map}
- id: charge_a
  name: Şarj A
  kind: charger
  station_id: charger_a
```

Dairesel zone merkezi dock_pose konumudur. Charger lokasyonu pose/kapsamı station_id üzerinden türetir; ikinci bir koordinat kaynağı tutmaz. Map endpoint image_url üretir; yerel dosya yolu istemciye açılmaz. Görsel/çözünürlük/origin aynı harita sürümüne aittir.

Topic / tip	Üretici ve sözleşme
/<id>/pose geometry_msgs/msg/PoseStamped	robot_pose_adapter, lokalizasyonun map -> <id>/base_link TF dönüşümünden üretir; header.frame_id=map, header.stamp=/clock. TF yok/bayatsa yeni geçerli pose yayınlanmaz.
/<id>/dock_contact std_msgs/msg/Bool	simulation_adapter, map pozunun atanmış istasyon dock_pose konumuna contact_radius ve contact_yaw_tolerance içinde olmasından türetir. Bool header taşımaz; alım zamanı izlenir. Bayat/eksik pose false üretir.
/<id>/odom nav_msgs/msg/Odometry	Robot sürüş/odometri adaptörü; hız ve ham mesafe.
/<id>/battery_state sensor_msgs/msg/BatteryState	battery_simulator; percentage ve power_supply_status.

Yeni alanların doğrulama ve sınıfları

Alan grubu	Doğrulama / uygulama
battery.policy	$0 \leq \text{emergency} < \text{low} < \text{idle_charge_level} \leq \text{release} \leq 1$; $\text{emergency} \leq \text{safety_reserve} \leq 1$. dock/charge_timeout >0. Live; yeni karar/şarj oturumu yakalar.
assignment	uncertainty ≥ 0 ; D_ref/S_ref/T_ref/L_ref >0; workload_window >0; hepsi sonlu. Live; yeni atama kararı.
tasks retry/requeue	max_requeues ve max_navigation_retries tam sayı ≥ 0 ; backoff/cooldown ≥ 0 . Live; aktif retry/requeue dizisinin bütçesi korunur.
navigation recovery_cooldown	Sonlu ≥ 0 simülasyon saniyesi. Live; yeni geçici hata oturumu yakalar. navigation_timeout >0; yeni goal yakalar.
battery.docking	Doğrusal eşik m/s, açısal eşik rad/s; ≥ 0 , stable_duration >0. Restart; simülatör/adaptör aynı sürümü kullanır.
fleet.health / fleet.web	Aralık/bayatlık >0, heartbeat_timeout > heartbeat_interval; Hz >0, kapasiteler pozitif tam sayı. Restart; producer/consumer birlikte geçer.
fleet.map / stations / locations	Sonlu pose/origin; resolution >0; benzersiz id; kapasite pozitif tam sayı; zone.radius >0; $0 < \text{contact_radius} \leq \text{zone.radius}$; yaw toleransı (0, pi]. Restart.
Referans bütünlüğü	kind pickup/dropoff/charger; map frame uyumu; station_id/profil/model çözülmeli. Konumlar harita ve footprint ile geçerli olmalı. Restart.

Tüm sayısal alanlar NaN/sonsuz değerleri reddeder. Charge timeout başlangıç SoC, charge_per_sec ve release hedefiyle erişilebilirlik açısından kontrol edilir; kasıtlı arıza deneyleri açık senaryo bayrağıyla ayrılır. charge_per_sec=0 normal şarj profili için reddedilir.

Robot IDLE ve fırsat şarjı koşulu sağlandığında yeni şarj oturumu açılır. Aktif oturum politika sürümünü korur. Live Apply aktif teslimatı kesmez; yalnız Restart-Apply aktif teslimat/yük güvenlik kontrolüne tabidir. Restart sonrasında QUEUED görevler kalır, manager_epoch yenilenir.

Özet şema + bu iki şema sayfası + bu doğrulama tablosu tek sözleşmedir. Ek alanlar yalnız açıklamada kullanılan serbest parametreler değildir; backend şemasında tip, birim ve uygulama sınıfıyla tanımlanır.